UBMA Innovation Incubator Platform

A modern, full-stack web application for managing university innovation incubator operations at Université Badji Mokhtar Annaba (UBMA), Algeria.



🎯 Project Summary

This platform serves as the digital backbone for UBMA's innovation incubator, facilitating the complete lifecycle of entrepreneurial projects from initial registration through incubation to acceleration. It features:

- **Anonymous Project Registration** - Students can register projects without creating accounts
- **Multi-Role Dashboard** - Tailored interfaces for students, supervisors, coaches, admins, and super admins
- **AI-Powered Assistant** - Integrated chatbot powered by n8n workflows for 24/7 support
- **Comprehensive Management** - Full CRUD operations for users, projects, juries, faculties, and more
- **Enterprise Security** - JWT authentication, device fingerprinting, rate limiting, and real-time monitoring
- **Bilingual Support** - Arabic and English throughout the platform

Current Status: ✅ Production-ready, deployed at https://incubator.onrender.com

Tech Stack: Vue 3 + Vite | Express 5 + Prisma | PostgreSQL | n8n AI | Telegram Monitoring

📋 Table of Contents

- Overview
- Features
- Technology Stack
- Project Structure
- Feature Overview by Role
- Prerequisites
- Quick Start
- Installation
- Configuration
- Running the Application
- API Documentation
- Database Schema
- Authentication & Authorization
- Frontend Architecture
- Backend Architecture
- Security Features
- Environment Variables
- Deployment
- Troubleshooting
- Developer Notes
- Technology Decisions

- License
- Support
- Author

🎯 Overview

The Badji Mokhtar University Innovation Incubator platform is designed to streamline the management of student entrepreneurship projects, provide mentorship opportunities, and facilitate the journey from idea to successful startup. The platform connects students, supervisors, and administrators in a collaborative environment.

Key Objectives

- **Project Management**: Register and track innovation projects
- **User Management**: Handle multiple user roles (students, supervisors, admins)
- **AI Assistant**: Integrated chatbot for real-time assistance
- **Faculty Integration**: Connect projects with academic departments
- **Jury System**: Manage project evaluations and jury members

---

✨ Features

For Students

- Anonymous project registration with team member details
- View registration status and updates
- AI-powered assistance via integrated chatbot
- Access to faculty and department information

For Supervisors

- Assigned to projects during registration
- Monitor project progress and status
- Collaborate with project teams

For Administrators (Admin Role: 90)

- View and manage project registrations
- User management with role assignment
- Approve or reject project submissions
- Access to system reports

For Super Administrators (SuperAdmin Role: 100)

- Complete system control and configuration
- Faculty and department management
- FAQ management for website
- Jury management and member assignment
- Location and partner management

- Acceleration and pre-incubation status configuration
- Full user and project oversight

General Features

- **Multi-role Authentication**: JWT dual-token system (access + refresh tokens)
- **AI Chat Integration**: n8n-powered chatbot with session persistence and markdown support
- **Device Fingerprinting**: FingerprintJS for enhanced security
- **Telegram Notifications**: Real-time alerts for critical system events and security issues
- **Responsive Design**: Modern green-themed UI with smooth gradients
- **Advanced Security**: Rate limiting, CORS, Helmet.js, input sanitization, XSS protection
- **Geo-Location Tracking**: IP-based location tracking for security monitoring
- **Comprehensive Logging**: Request, error, security, and unauthorized access logs
- **Data Validation**: Client and server-side validation with detailed error messages

---

🛠️ Technology Stack

Frontend

- **Framework**: Vue.js 3.5.x (Composition API with script setup)
- **Build Tool**: Vite 7.x
- **Routing**: Vue Router 4.6.x
- **State Management**: Pinia 3.x
- **Styling**: Scoped CSS with green-themed design system
- **HTTP Client**: Axios 1.13.x with interceptors
- **Markdown**: Marked.js 17.x for AI chat rendering
- **Security**: DOMPurify 3.3.x for XSS protection
- **Device Fingerprinting**: FingerprintJS 5.x for device identification

Backend

- **Runtime**: Node.js
- **Framework**: Express.js 5.x
- **ORM**: Prisma 6.x with PostgreSQL
- **Database**: PostgreSQL
- **Authentication**: JWT (Access & Refresh Tokens) with bcrypt 6.x
- **Security**:
  - Helmet.js 8.x (HTTP headers)
  - CORS 2.x with whitelist
  - Express Rate Limit 7.x (DDoS protection)
  - Cookie Parser 1.x
  - Compression 1.x
- **Logging**: Custom file-based logging system (reqLog, errLog, securityLog, unauthorizedReqLog)
- **Notifications**: Telegram Bot API for real-time alerts
- **AI Integration**: Axios for n8n webhook communication
- **Utilities**: UUID 11.x, Lodash 4.x, date-fns 4.x, ms 2.x
- **File Uploads**: Multer 2.x

- **Cloud Storage**: Cloudinary 2.x

Development Tools

- **Package Manager**: npm
- **API Testing**: Built-in REST endpoints
- **Code Quality**: ESLint compatible
- **Version Control**: Git

📁 Project Structure

```
incubator/
├── backend/
│   ├── config/
│   │   ├── allowedOrigins.js       # CORS configuration
│   │   ├── compressionConfig.js    # Response compression
│   │   ├── corsConfig.js           # CORS options
│   │   ├── db.js                   # Database connection
│   │   ├── headersConfig.js        # Custom headers
│   │   ├── helmetConfig.js         # Security headers
│   │   ├── limiterConfig.js        # Rate limiting
│   │   ├── notifier.js             # Email notifications
│   │   └── rolesList.js            # User role definitions
│   ├── controllers/
│   │   ├── accelerationStatusController.js
│   │   ├── aiControllers.js        # AI chatbot logic
│   │   ├── authController.js       # Authentication
│   │   ├── facultiesController.js
│   │   ├── faqController.js
│   │   ├── juryController.js
│   │   ├── juryMembreController.js
│   │   ├── juryMembreRoleController.js
│   │   ├── locationsController.js
│   │   ├── logoutController.js
│   │   ├── partnersController.js
│   │   ├── preIncubationStatusController.js
│   │   ├── projectsController.js
│   │   ├── refreshTokenController.js
│   │   ├── registerController.js
│   │   ├── registrationControllers.js
│   │   ├── usersController.js
│   │   └── userStatusController.js
│   ├── logs/
│   │   ├── errLog.txt              # Error logs
│   │   ├── reqLog.txt              # Request logs
│   │   ├── securityLog.txt         # Security events
│   │   └── unauthorizedReqLog.txt  # Unauthorized access
│   ├── middlewares/
│   │   ├── errorHandler.js         # Global error handling
│   │   ├── logEvents.js            # Event logging
│   │   ├── optionalJwt.js          # Optional authentication
```

```
│   │   ├── refreshOnRequest.js     # Token refresh
│   │   ├── secureMiddleware.js     # Security middleware
│   │   ├── verifyJWT.js            # JWT verification
│   │   └── verifyRoles.js          # Role-based access
│   ├── models/
│   │   ├── accelerationStatusModels.js
│   │   ├── FacultiesModels.js
│   │   ├── faqModels.js
│   │   ├── juryMembreModels.js
│   │   ├── juryMembreRoleModels.js
│   │   ├── juryModels.js
│   │   ├── locationsModels.js
│   │   ├── partnersModels.js
│   │   ├── preIncubationStatusModels.js
│   │   ├── projectsModels.js
│   │   ├── registrationModels.js
│   │   ├── tokensModels.js
│   │   └── UsersModels.js
│   ├── prisma/
│   │   └── schema.prisma           # Database schema
│   ├── routes/
│   │   ├── accelerationStatusRoutes.js
│   │   ├── aiRoutes.js
│   │   ├── authRoutes.js
│   │   ├── facultiesRoutes.js
│   │   ├── faqRoutes.js
│   │   ├── juryMembreRoleRoutes.js
│   │   ├── juryMembreRoutes.js
│   │   ├── juryRoutes.js
│   │   ├── locationsRoutes.js
│   │   ├── partnersRoutes.js
│   │   ├── preIncubationStatusRoutes.js
│   │   ├── projectsRoutes.js
│   │   ├── registrationRoutes.js
│   │   └── usersRoutes.js
│   ├── utils/
│   │   ├── fingerprintUtils.js     # Device fingerprinting
│   │   ├── getRealIP.js            # IP detection
│   │   └── tokensAndCookiesUtils.js # Token management
│   ├── package.json
│   └── server.js                   # Application entry point
│
├── frontend/
│   ├── public/
│   │   └── incubator_logo.png      # University logo
│   ├── src/
│   │   ├── assets/                 # Static assets
│   │   ├── components/
│   │   │   ├── dashboards/
│   │   │   │   ├── AccelerationStatusManagement.vue
│   │   │   │   ├── FacultiesManagement.vue
│   │   │   │   ├── FAQManagement.vue
│   │   │   │   └── ... (more dashboard components)
│   │   │   └── AiChat.vue          # AI Assistant component
```

```
│   │   ├── router/
│   │   │   └── index.js           # Vue Router configuration
│   │   ├── stores/
│   │   │   └── useAuthStore.js    # Authentication state
│   │   ├── utils/
│   │   │   └── getFingerprint.js  # Device fingerprinting
│   │   ├── views/
│   │   │   ├── Dashboard.vue      # Main dashboard
│   │   │   ├── Home.vue           # Landing page
│   │   │   ├── Login.vue          # Login page
│   │   │   ├── Register.vue       # Project registration
│   │   │   └── RegistrationSuccess.vue
│   │   ├── api.js                 # Axios configuration
│   │   ├── App.vue                # Root component
│   │   ├── main.js                # Application entry
│   │   └── style.css              # Global styles
│   ├── index.html
│   ├── package.json
│   └── vite.config.js
│
└── README.md
```

🔒 Feature Overview by Role

Feature	Guest	Student	Supervisor	Coach	Admin	SuperAdmin
View Public Pages	✅	✅	✅	✅	✅	✅
AI Chat Assistant	✅	✅	✅	✅	✅	✅
Register Project	✅	✅	✅	✅	✅	✅
Login/Logout	❌	✅	✅	✅	✅	✅
View Dashboard	❌	✅	✅	✅	✅	✅
Account Settings	❌	✅	✅	✅	✅	✅
View Assigned Projects	❌	✅	✅	✅	✅	✅
Manage Project Registrations	❌	❌	❌	❌	✅	✅
User Management	❌	❌	❌	❌	✅	✅
Faculty & Department Management	❌	❌	❌	❌	❌	✅
FAQ Management	❌	❌	❌	❌	❌	✅
Jury Management	❌	❌	❌	❌	❌	✅
Location & Partner Management	❌	❌	❌	❌	❌	✅
Status Configuration	❌	❌	❌	❌	❌	✅

Feature	Guest	Student	Supervisor	Coach	Admin	SuperAdmin
System Configuration	❌	❌	❌	❌	❌	✅

📋 Prerequisites

Before you begin, ensure you have the following installed:

- **Node.js**: Version 16.x or higher (recommended: 18.x or 20.x LTS)
- **npm**: Version 8.x or higher
- **PostgreSQL**: Version 13.x or higher (recommended: 15.x or 16.x)
- **Git**: Latest version

Optional (for full functionality):

- **n8n**: For AI chatbot integration (self-hosted or cloud)
- **Telegram Bot**: For real-time notifications
- **Cloudinary Account**: For file upload storage

🚀 Quick Start

For a rapid development setup:

```
# Clone and install
git clone https://github.com/Youcef-Berbague/incubator.git
cd incubator

# Backend setup
cd backend
npm install
cp .env.example .env  # Create and configure .env file
npx prisma migrate dev  # Setup database
npx prisma generate  # Generate Prisma client

# Terminal 1: Start backend
npm run dev

# Terminal 2: Start frontend
cd ../frontend
cp .env.example .env  # Create and configure .env file
npm install
npm run dev
```

Visit `http://localhost:5173` for the frontend and `http://localhost:3000/api` for the backend API.

🚀 Installation

1. Clone the Repository

```
git clone https://github.com/Youcef-Berbague/incubator.git
cd incubator
```

2. Backend Setup

```
cd backend
npm install
```

Dependencies installed:

- Express 5.x, Prisma 6.x, bcrypt 6.x, jsonwebtoken 9.x
- helmet 8.x, cors 2.x, express-rate-limit 7.x
- axios 1.x, uuid 11.x, cloudinary 2.x, multer 2.x
- nodemon 3.x (dev dependency)

3. Frontend Setup

```
cd ../frontend
npm install
```

Dependencies installed:

- Vue 3.5.x, Vue Router 4.6.x, Pinia 3.x
- Vite 7.x, axios 1.13.x
- marked 17.x, dompurify 3.3.x, @fingerprintjs/fingerprintjs 5.x

---

⚙️ Configuration

Database Setup

1. Create a PostgreSQL database:

```
CREATE DATABASE incubator_db;
```

2. Configure environment variables (see Environment Variables)

3. Run Prisma migrations:

```
cd backend
npx prisma migrate dev
npx prisma generate
```

Environment Variables

Create a `.env` file in the `backend` directory:

```
# Database
DATABASE_URL="postgresql://username:password@localhost:5432/incubator_db"

# JWT Secrets (use strong random strings)
ACCESS_TOKEN_SECRET="your-access-token-secret-here"
REFRESH_TOKEN_SECRET="your-refresh-token-secret-here"

# Server Configuration
PORT=3000
NODE_ENV=development

# Telegram Notifications
TELEGRAM_BOT_PRINTYDZ_NOTIFIER="your-telegram-bot-token"
TELEGRAM_CHAT_ID="your-telegram-chat-id"

# n8n AI Integration
N8N_CHAT_WEBHOOK_URL="https://your-n8n-instance.com/webhook/chat"
N8N_API_KEY="your-n8n-api-key"

# Cloudinary (for file uploads)
CLOUDINARY_CLOUD_NAME="your-cloud-name"
CLOUDINARY_API_KEY="your-api-key"
CLOUDINARY_API_SECRET="your-api-secret"
```

Create a `.env` file in the `frontend` directory:

```
VITE_BACKEND_BASEURL=http://localhost:3000/api
```

🏃 Running the Application

Development Mode

Backend:

```
cd backend
npm run dev
# Server runs on http://localhost:3000 (or PORT in .env)
```

Frontend:

```

```

```
cd frontend
npm run dev
# Application runs on http://localhost:5173
```

Production Mode

Backend:

```
cd backend
npm start
```

Frontend:

```
cd frontend
npm run build
npm run preview
```

📡 API Documentation

Base URL

```
http://localhost:3000/api
```

Authentication Endpoints

Check Authentication Status

```
GET /auth/status
Cookie: jwt=xxx
```

Response:

```
{
  "authenticated": true
}
```

Login

```
POST /auth/login
Content-Type: application/json
X-Fingerprint: {device-fingerprint-json}

{
  "email": "user@example.com",
  "password": "SecurePassword123",
  "rememberMe": false
}
```

Response:

```
{
  "status": "success",
  "message": "Login with success"
}
```

Note:

- Refresh token is set as HTTP-only cookie
- Access token is managed automatically by middleware
- Device fingerprint is required for security
- `rememberMe: true` extends refresh token to 7 days (default: 4 hours)

Refresh Token

```
GET /auth/refresh
Cookie: jwt=xxx
X-Fingerprint: {device-fingerprint-json}
```

Note: This endpoint is called automatically by the frontend interceptor when access token expires

Logout

```
POST /auth/logout
Cookie: refreshToken=xxx
```

Project Registration Endpoints

Register Project (Public - No Authentication Required)

```
POST /api/projects/register
Content-Type: application/json
```

```
{
  "projectTitle": "AI-Powered Agriculture System",
  "numberOfStudents": 2,
  "numberOfSupervisors": 1,
  "facultyId": 1,
  "departmentId": 5,
  "students": [
    {
      "firstName": "Ahmed",
      "lastName": "Benali",
      "email": "ahmed.benali@student.univ-annaba.dz",
      "phone": "0555123456",
      "academicLevel": "M2"
    }
  ],
  "supervisors": [
    {
      "firstName": "Karim",
      "lastName": "Meziane",
      "email": "k.meziane@univ-annaba.dz",
      "phone": "0661234567"
    }
  ]
}
```

Validation Rules:

- Project title: 5-255 characters
- Number of students: 1-6
- Number of supervisors: 1-3
- Academic levels: L1, L2, L3, M1, M2, PhD (validated against whitelist)
- Phone: 10 digits (Algerian format: 05XXXXXXXX or 06XXXXXXXX or 07XXXXXXXX)
- Email format validated with regex: `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`
- Names: 2-100 characters, letters only (supports Arabic, French, English with spaces, hyphens, apostrophes)
- Faculty and department validation:
  - Faculty must exist in database
  - Department must belong to selected faculty
  - Automatic verification via foreign key constraints
- Duplicate title check (case-insensitive)
- Array length validation (students/supervisors count must match arrays)

Registration Workflow:

1. **User fills form** (anonymous, no login required)
2. **Client-side validation** (Vue form validation)
3. **Submit to** `/api/projects/register`
4. **Server-side validation** (comprehensive error array)
5. **Faculty/Department verification** (database lookup)

6. **Duplicate title check** (existing registrations)
7. **Transaction creation:**
    - Create project_registrations record (status: "pending")
    - Create registration_students records (with CASCADE delete)
    - Create registration_supervisors records (with CASCADE delete)
8. **Success response** with registration ID
9. **Redirect to success page** with confirmation
10. **Admin review** (Approve/Reject via dashboard)

Approval Process (Admin/SuperAdmin):

- **Pending**: Initial state after submission
- **Approved**: Admin approves → Can trigger project creation in main project table
- **Rejected**: Admin rejects → Registration archived or deleted

Get All Registrations (Admin/SuperAdmin)

```
GET /api/registrations
Cookie: jwt=xxx
```

Get Registration by ID (Admin/SuperAdmin)

```
GET /api/registrations/:id
Cookie: jwt=xxx
```

Update Registration Status (Admin/SuperAdmin)

```
PATCH /api/registrations/:id/status
Cookie: jwt=xxx
Content-Type: application/json

{
  "status": "approved"
}
```

Status Options: pending, approved, rejected

Delete Registration (Admin/SuperAdmin)

```
DELETE /api/registrations/:id
Cookie: jwt=xxx
```

Faculty & Department Endpoints (Public)

Get All Faculties

```
GET /api/faculties
```

Response:

```
[
  {
    "id": 1,
    "faculty_name": "كلية العلوم الدقيقة",
    "faculty_name_en": "Faculty of Exact Sciences",
    "departements": [
      {
        "id": 1,
        "departement_name": "قسم الإعلام الآلي",
        "departement_name_en": "Computer Science Department"
      }
    ]
  }
]
```

Get Single Faculty

```
GET /api/faculties/:id
```

Create Faculty (SuperAdmin Only)

```
POST /api/faculties
Cookie: jwt=xxx
Content-Type: application/json

{
  "faculty_name": "كلية العلوم الدقيقة",
  "faculty_name_en": "Faculty of Exact Sciences"
}
```

Update Faculty (SuperAdmin Only)

```
PATCH /api/faculties/:id
Cookie: jwt=xxx
```

```
Content-Type: application/json

{
  "faculty_name": "Updated Name"
}
```

Delete Faculty (SuperAdmin Only)

```
DELETE /api/faculties/:id
Cookie: jwt=xxx
```

User Management (Authenticated)

Get Current User

```
GET /api/users/me
Cookie: jwt=xxx
```

Response:

```
{
  "id": 1,
  "email": "user@example.com",
  "first_name": "John",
  "last_name": "Doe",
  "phone_number": "0555123456",
  "role": [100],
  "status": "active",
  "academic_level": "M2",
  "departement_id": 1,
  "createdate": "2025-01-01T00:00:00.000Z"
}
```

Get All Users (Admin/SuperAdmin)

```
GET /api/users
Cookie: jwt=xxx
```

Get User by ID (Admin/SuperAdmin)

```

```

```
GET /api/users/:id
Cookie: jwt=xxx
```

Update User (Admin/SuperAdmin)

```
PATCH /api/users/:id
Cookie: jwt=xxx
Content-Type: application/json

{
  "first_name": "Updated Name",
  "role": [90],
  "status": "active"
}
```

Delete User (SuperAdmin Only)

```
DELETE /api/users/:id
Cookie: jwt=xxx
```

AI Chat Endpoints

Send Message to AI Assistant

```
POST /api/ai/chat
Content-Type: application/json

{
  "message": "What is the incubator program?",
  "sessionId": "chat_1234567890_abc123"
}
```

Response:

```
{
  "reply": "مرحباً! حاضنة جامعة باجي مختار عنابة هي...",
  "sessionId": "chat_1234567890_abc123"
}
```

Note:

- No authentication required for AI chat

- SessionId is auto-generated if not provided
- Messages are sent to n8n webhook for processing
- Supports Arabic and English responses
- Session persists across page reloads via localStorage

Public API Endpoints (No Authentication)

Get All FAQs

```
GET /api/faqs
```

Get All Locations

```
GET /api/locations
```

Get All Partners

```
GET /api/partners
```

Get All Acceleration Statuses

```
GET /api/acceleration-statuses
```

Get All Pre-Incubation Statuses

```
GET /api/pre-incubation-statuses
```

Jury Management Endpoints (SuperAdmin)

Get All Juries

```
GET /api/juries
Cookie: jwt=xxx
```

Create Jury

```
POST /api/juries
Cookie: jwt=xxx
Content-Type: application/json

{
  "jury_code": "JURY2025-001",
  "date": "2025-12-31T10:00:00.000Z",
  "location_id": 1,
  "faculty_id": 1
}
```

Get All Jury Members

```
GET /api/jury-members
Cookie: jwt=xxx
```

Get All Jury Member Roles

```
GET /api/jury-member-roles
Cookie: jwt=xxx
```

Projects Endpoints (Authenticated - SuperAdmin, Admin, Coach, Supervisor)

Get All Projects

```
GET /api/projects
Cookie: jwt=xxx
```

Get Project by ID

```
GET /api/projects/:id
Cookie: jwt=xxx
```

🗄️ Database Schema

Core Tables

users

- `id`: Primary key (auto-increment)

- `email`: Unique email address (VARCHAR 100)
- `phone_number`: Unique phone number (VARCHAR 10)
- `password`: Hashed password with bcrypt (VARCHAR 255)
- `first_name`: User's first name (VARCHAR 30)
- `last_name`: User's last name (VARCHAR 40)
- `role`: Array of role codes (Int[])
- `status`: Account status (VARCHAR 100)
- `academic_level`: Academic level (VARCHAR 50) - L1, L2, L3, M1, M2, PhD
- `p_team_id`: Foreign key to project_team
- `departement_id`: Foreign key to departements
- `createdate`: Account creation timestamp
- `updatedate`: Last update timestamp

project

- `id`: Primary key (auto-increment)
- `title`: Project title (VARCHAR 250)
- `project_code`: Unique project code (VARCHAR 100)
- `finalpv`: Final evaluation document path (VARCHAR 1000)
- `incubated`: Boolean flag for incubation status
- `progress_percent`: Project completion percentage (SmallInt)
- `team_id`: Foreign key to project_team
- `pre_incubation_status`: Foreign key to pre_incubation_status
- `acceleration_status`: Foreign key to acceleration_status
- `departement_id`: Foreign key to departements

faculties

- `id`: Primary key (auto-increment)
- `faculty_name`: Faculty name in Arabic (VARCHAR 60)
- `faculty_name_en`: Faculty name in English (VARCHAR 100)

departements

- `id`: Primary key (auto-increment)
- `departement_name`: Department name in Arabic (VARCHAR 60)
- `departement_name_en`: Department name in English (VARCHAR 100)
- `faculty_id`: Foreign key to faculties

refresh_tokens

- `id`: Primary key (auto-increment)
- `token`: Hashed refresh token (bcrypt)
- `user_id`: Foreign key to users
- `uuid`: Unique token identifier
- `fingerprint`: Device fingerprint (JSON)
- `ip_address`: User's IP address
- `user_agent`: Browser user agent string

- `remembreme`: Boolean for extended session
- `request_count`: Number of refresh requests
- `created_at`: Token creation timestamp

project_registrations

- `id`: Primary key (auto-increment)
- `title`: Project title (VARCHAR 250)
- `faculty_id`: Foreign key to faculties
- `departement_id`: Foreign key to departements
- `status`: Registration status - pending, approved, rejected (VARCHAR 30)
- `submitted_at`: Submission timestamp

registration_students

- `id`: Primary key (auto-increment)
- `registration_id`: Foreign key to project_registrations
- `first_name`: Student first name (VARCHAR 30)
- `last_name`: Student last name (VARCHAR 40)
- `email`: Student email (VARCHAR 100)
- `phone`: Student phone (VARCHAR 20)
- `academic_level`: L1, L2, L3, M1, M2, PhD (VARCHAR 50)

registration_supervisors

- `id`: Primary key (auto-increment)
- `registration_id`: Foreign key to project_registrations
- `first_name`: Supervisor first name (VARCHAR 30)
- `last_name`: Supervisor last name (VARCHAR 40)
- `email`: Supervisor email (VARCHAR 100)
- `phone`: Supervisor phone (VARCHAR 20)

jury

- `id`: Primary key (auto-increment)
- `jury_code`: Unique jury code (VARCHAR 100)
- `date`: Jury session date
- `so_partener_id`: Foreign key to socio_economic_partner
- `location_id`: Foreign key to locations
- `faculty_id`: Foreign key to faculties

jury_membre

- `id`: Primary key (auto-increment)
- `membre_id`: Foreign key to users
- `role_id`: Foreign key to jury_membre_role

locations

- `id`: Primary key (auto-increment)
- `location_name`: Location name (VARCHAR 50)

socio_economic_partner

- `id`: Primary key (auto-increment)
- `full_name`: Partner name (VARCHAR 50)
- `phone_number`: Unique phone (VARCHAR 10)
- `email`: Unique email (VARCHAR 100)
- `professionel_adress`: Professional address (VARCHAR 150)

faq

- `id`: Primary key (auto-increment)
- `question`: FAQ question (VARCHAR 500)
- `answer`: FAQ answer (Text)
- `category`: Category name (VARCHAR 50)
- `keywords`: Search keywords (Text)
- `priority`: Display priority (Int, default: 0)
- `isactive`: Active status (Boolean, default: true)
- `createdat`: Creation timestamp
- `updatedat`: Update timestamp

acceleration_status

- `id`: Primary key (auto-increment)
- `acceleration_name`: Status name (VARCHAR 30)

pre_incubation_status

- `id`: Primary key (auto-increment)
- `status_name`: Status name (VARCHAR 30)

🔐 Authentication & Authorization

JWT Token System

The application uses a sophisticated dual-token authentication system with device fingerprinting:

1. **Access Token**
    - Short-lived (15 minutes)
    - Generated automatically on each request via `refreshOnRequest` middleware
    - Stored in `req.accessToken` (server-side)
    - Contains: `{ userInfos: { userId, roles } }`
    - Verified by `verifyJWT` middleware
2. **Refresh Token**

- Long-lived: 7 days (with rememberMe) or 4 hours (without)
- Stored as HTTP-only, Secure, SameSite=Strict cookie named `jwt`
- Hashed with bcrypt before storing in database
- Contains: `{ userInfos: { userId }, id: tokenUUID }`
- Tied to device fingerprint (FingerprintJS)
- Tracked with IP address, user agent, and request count

3. **Device Fingerprinting**
   - Uses FingerprintJS (@fingerprintjs/fingerprintjs)
   - Sent as `x-fingerprint` header with each request
   - Stored as JSON in refresh_tokens table
   - Provides additional security layer against token theft

4. **Token Rotation**
   - Old refresh token is deleted on login
   - New refresh token generated for each login session
   - UUID tracking prevents token reuse
   - Automatic cleanup on logout

User Roles

Roles are stored as numeric codes in an integer array:

- **100 - SuperAdmin**: Full system access, all configurations
- **90 - Admin**: User and registration management
- **70 - Coach**: Project mentoring and guidance
- **60 - Supervisor**: Assigned project supervision
- **40 - Student**: Project team member
- **10 - Guest**: Limited public access

Note: Users can have multiple roles simultaneously (e.g., [60, 40] = Supervisor + Student)

Role-Based Access Control (RBAC)

```
const ROLES_LIST = require("./config/rolesList");
const verifyJWT = require("./middlewares/verifyJWT");
const verifyRoles = require("./middlewares/verifyRoles");

// Example: Protecting routes
router.get(
  "/admin-only",
  verifyJWT, // Verify JWT token
  verifyRoles(ROLES_LIST.admin, ROLES_LIST.superAdmin), // Allow multiple
roles
  controller.adminAction
);

// Example: SuperAdmin only
```

```
router.post(
  "/faculties",
  verifyJWT,
  verifyRoles(ROLES_LIST.superAdmin),
  controller.createFaculty
);
```

Role Verification Logic

- `verifyJWT`: Validates access token and extracts user roles
- `verifyRoles(...allowedRoles)`: Checks if user has at least one of the allowed roles
- Returns 401 if user lacks required permissions

🎨 Frontend Architecture

Vue Router Configuration

```
const routes = [
  { path: "/", component: Home },
  { path: "/login", component: Login, meta: { requiresGuest: true } },
  { path: "/register", component: Register },
  { path: "/registration-success", component: RegistrationSuccess },
  { path: "/dashboard", component: Dashboard, meta: { requiresAuth: true }
},
  {
    path: "/dashboard/registrations",
    component: RegistrationsManagement,
    meta: { requiresAuth: true, requiresAdmin: true },
  },
  {
    path: "/dashboard/users",
    component: UsersManagement,
    meta: { requiresAuth: true, requiresAdmin: true },
  },
  {
    path: "/dashboard/faculties",
    component: FacultiesManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
  {
    path: "/dashboard/account",
    component: MyAccountSettings,
    meta: { requiresAuth: true },
  },
  {
    path: "/dashboard/faqs",
    component: FAQManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
  {
```

```
    path: "/dashboard/acceleration-statuses",
    component: AccelerationStatusManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
  {
    path: "/dashboard/locations",
    component: LocationsManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
  {
    path: "/dashboard/partners",
    component: PartnersManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
  {
    path: "/dashboard/pre-incubation-statuses",
    component: PreIncubationStatusManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
  {
    path: "/dashboard/jury-member-roles",
    component: JuryMemberRolesManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
  {
    path: "/dashboard/juries",
    component: JuriesManagement,
    meta: { requiresAuth: true, requiresSuperAdmin: true },
  },
];
```

Navigation Guards:

```
router.beforeEach(async (to, from, next) => {
  const authStore = useAuthStore();

  // Initialize auth state
  if (!authStore.isInitialized) {
    await authStore.checkAuth();
  }

  // Redirect authenticated users away from login
  if (to.meta.requiresGuest && authStore.isAuthenticated) {
    return next("/dashboard");
  }

  // Require authentication for protected routes
  if (to.meta.requiresAuth && !authStore.isAuthenticated) {
    return next("/login");
  }

  // Check role-based access
```

```
  if (to.meta.requiresSuperAdmin) {
    const userRole = Math.max(...(authStore.user?.role || [0]));
    if (userRole < 100) return next("/dashboard");
  }

  if (to.meta.requiresAdmin) {
    const userRole = Math.max(...(authStore.user?.role || [0]));
    if (userRole < 90) return next("/dashboard");
  }

  next();
});
```

API Client Configuration (Axios)

Base Configuration:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BACKEND_BASEURL, //
http://localhost:3000/api
  withCredentials: true, // Send cookies with requests
});
```

Request Interceptor (Device Fingerprinting):

```
api.interceptors.request.use(async (config) => {
  // Load fingerprint once per session
  if (!cachedFingerprint) {
    cachedFingerprint = await loadFingerprint();
  }

  // Add fingerprint to headers
  if (cachedFingerprint) {
    config.headers["x-fingerprint"] = JSON.stringify(cachedFingerprint);
  }

  return config;
});
```

Response Interceptor (Automatic Token Refresh):

```
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;
    const status = error.response?.status;
```

```
    // Skip auth endpoints
    const authEndpoints = ["/auth/refresh", "/auth/login",
"/auth/register"];
    if (authEndpoints.some((e) => originalRequest.url?.includes(e))) {
      return Promise.reject(error);
    }

    // Handle 401 - attempt token refresh
    if (status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;

      if (!isRefreshing) {
        isRefreshing = true;
        refreshPromise = api
          .get("/auth/refresh")
          .then(() => console.log("Token refreshed"))
          .catch(() => {
            authStore.user = null; // Logout on refresh failure
            throw new Error("Session expired");
          })
          .finally(() => {
            isRefreshing = false;
            refreshPromise = null;
          });
      }

      await refreshPromise;
      return api(originalRequest); // Retry original request
    }

    return Promise.reject(error);
  }
);
```

State Management (Pinia)

Authentication Store:

```
export const useAuthStore = defineStore("auth", {
  state: () => ({
    user: null, // Current user object
    isLoading: false, // Loading state for async operations
    error: null, // Error message
    isInitialized: false, // Auth check completed
  }),

  getters: {
    isAuthenticated: (state) => !!state.user,
    userRole: (state) => state.user?.role || null, // Array of role codes
    userName: (state) => {
      if (!state.user) return null;
```

```
      return `${state.user.first_name} ${state.user.last_name}`.trim();
    },
    userEmail: (state) => state.user?.email || null,
  },

  actions: {
    async login(email, password, rememberMe = false) {
      this.isLoading = true;
      this.error = null;
      try {
        await api.post("/auth/login", { email, password, rememberMe });
        const userRes = await api.get("/users/me");
        this.user = userRes.data;
        return true;
      } catch (err) {
        this.error = err.response?.data?.message || "Login failed";
        return false;
      } finally {
        this.isLoading = false;
      }
    },

    async checkAuth() {
      if (this.isInitialized) return;
      this.isLoading = true;
      try {
        const status = await api.get("/auth/status");
        if (status.data.authenticated) {
          const userRes = await api.get("/users/me");
          this.user = userRes.data;
        } else {
          this.user = null;
        }
      } catch (err) {
        this.user = null;
      } finally {
        this.isInitialized = true;
        this.isLoading = false;
      }
    },

    async logout() {
      try {
        await api.post("/auth/logout");
      } catch (err) {
        console.warn("Logout failed");
      } finally {
        this.user = null;
        this.error = null;
        return true;
      }
    },

    clearError() {
```

```
        this.error = null;
      },
    },
  });
```

Component Structure

- **Views**: Page-level components (Home, Login, Register, Dashboard, RegistrationSuccess)
- **Components**: Reusable UI components (AiChat.vue)
- **Dashboards**: Role-specific management components
  - SuperAdminDashboard.vue - Main dashboard for superadmin
  - RegistrationsManagement.vue - Project registration management
  - UsersManagement.vue - User CRUD operations
  - FacultiesManagement.vue - Faculty and department management
  - FAQManagement.vue - FAQ content management
  - JuriesManagement.vue - Jury session management
  - JuryMemberRolesManagement.vue - Jury role management
  - LocationsManagement.vue - Location management
  - PartnersManagement.vue - Socio-economic partner management
  - AccelerationStatusManagement.vue - Acceleration status configuration
  - PreIncubationStatusManagement.vue - Pre-incubation status configuration
  - MyAccountSettings.vue - User profile settings

Design System

Color Palette:

- Primary Green: `#61ce70` (buttons, highlights, chat UI)
- Secondary Green: `#23a455` (gradients, accents)
- Light Green: `#f0fdf4` (backgrounds)
- Green Borders: `#e8f5eb`
- Tertiary Red: `#ff6b6b` (errors, warnings)
- Gray: `#54595f` (text, borders)
- Black: `#000000` (footer, dark sections)
- White: `#ffffff` (backgrounds, text on dark)

Typography:

- Font Family: Inter, system fonts
- Responsive font sizes
- Clear hierarchy

AI Chat Component Details

Integration Architecture:

```
User Input → AiChat.vue → Axios → Backend → n8n Webhook → AI Model →
Response
```

Session Management:

- Session IDs auto-generated: `chat_{timestamp}_{random}`
- Stored in localStorage for persistence across page reloads
- Chat history stored locally in JSON format
- Session context maintained server-side in n8n

Message Flow:

1. User types message in input field
2. Message added to local messages array with sender: "user"
3. POST request sent to `/api/ai/chat` with message and sessionId
4. Backend forwards to n8n webhook with API key authentication
5. n8n processes message through AI workflow
6. AI response returned to backend
7. Backend sends response to frontend
8. Response displayed with typewriter effect (5ms per character)
9. Messages saved to localStorage

Markdown Support:

- **marked.js** for parsing markdown syntax
- **DOMPurify** for XSS sanitization
- Allowed HTML tags: p, br, hr, a, strong, em, h1-h6, ul, ol, li, blockquote, pre, code
- Safe link handling with href, target, rel attributes

UI Features:

- Floating chat button (60px circle) with green gradient
- Slide-up animation for chat container (380x550px)
- User messages: right-aligned, green gradient bubbles
- Bot messages: left-aligned, white background with green border
- Typing indicator: animated green dot
- Custom green scrollbar
- Auto-scroll to bottom on new messages
- Welcome message in Arabic on first open
- Chat history preservation across sessions

🏗️ Backend Architecture

Middleware Stack (Execution Order)

1. **Trust Proxy** - `app.set("trust proxy", 1)` for Nginx/Render deployment
2. **Security Middleware** (Applied First)
    - `helmet` - HTTP security headers
    - `headersConfig` - Custom security headers

- `compression` - Response compression (gzip/deflate)
- `limiter` - Global rate limiting (500 req/10min)

3. **CORS Configuration** - `cors(corsOptions)` with whitelist
4. **Built-in Express Middleware**
   - `express.urlencoded({ extended: false })` - Form data parsing
   - `express.json({ limit: "150kb" })` - JSON body parsing with size limit
   - `express.static` - Static file serving from /public
5. **Cookie Parser** - `cookie-parser()` for reading cookies
6. **Logging Middleware**
   - `logger` - Custom request/error logging to files
7. **Security Routing Middleware**
   - `secureMiddleware` - Additional request sanitization
8. **Intrusion Detection Middleware**
   - `refreshOnRequest()` - Automatic access token refresh
   - `optionalJWT` - Flexible JWT verification for public/private routes
9. **Route-Specific Middleware**
   - `verifyJWT` - Required authentication for protected routes
   - `verifyRoles(...roles)` - Role-based access control
   - `loginLimiter` - Login-specific rate limiting (10 req/15min)
   - `registerLimiter` - Registration rate limiting (5 req/hour)
10. **Error Handling** (Applied Last)
    - `errorHandler` - Global error handler with logging

Controller Pattern

```
const getResource = async (req, res) => {
  try {
    const data = await Model.findMany();
    res.json(data);
  } catch (error) {
    res.status(500).json({
      message: "Server error",
      error: error.message,
    });
  }
};
```

Model Layer (Prisma)

```
const prisma = new PrismaClient();

// Example query
const users = await prisma.user.findMany({
  where: { role: "USER" },
  include: { projects: true },
});
```

🔒 Security Features

Implemented Security Measures

1. **HTTP Security Headers** (Helmet.js 8.x)
    - Content Security Policy (CSP)
    - X-Frame-Options (clickjacking protection)
    - X-Content-Type-Options (MIME sniffing prevention)
    - HSTS (Strict-Transport-Security)
    - X-Download-Options
    - X-Permitted-Cross-Domain-Policies
2. **CORS Protection**
    - Whitelist-based origins:
        - https://incubator.onrender.com
        - http://localhost:5173
        - http://localhost:4173
        - http://localhost:3000
    - Credentials support enabled
    - Pre-flight handling
    - Custom allowed headers
3. **Advanced Rate Limiting**
    - Global limiter: 500 requests per 10 minutes
    - Login limiter: 10 attempts per 15 minutes (per email)
    - Registration limiter: 5 attempts per hour (per IP)
    - Telegram alerts on limit breaches
    - Geo-location tracking for suspicious IPs
    - Skips localhost in development
4. **Input Validation**
    - Server-side validation with detailed error messages
    - Client-side Vue form validation
    - Prisma ORM prevents SQL injection

- Name validation: letters, spaces, hyphens, apostrophes only
- Email format validation
- Phone number format (10 digits Algerian)
- Academic level whitelist

5. **XSS Protection**

- DOMPurify 3.3.x sanitization in AI chat
- Marked.js for safe markdown rendering
- CSP headers restrict inline scripts
- HTML entity encoding

6. **Authentication Security**

- Password hashing: bcrypt 6.x with salt rounds
- JWT access tokens (15 min expiry)
- JWT refresh tokens (4h-7d expiry)
- Device fingerprinting (FingerprintJS 5.x)
- Token rotation on login
- Refresh token hashed in database
- UUID tracking for token uniqueness
- IP address and user agent logging

7. **Cookie Security**

- HTTP-only flag (no JavaScript access)
- Secure flag in production (HTTPS only)
- SameSite=Strict (CSRF protection)
- Path=/ for refresh tokens
- Auto-clear on logout

8. **Logging & Monitoring**

- Request logging (reqLog.txt)
- Error logging (errLog.txt)
- Security events (securityLog.txt)
- Unauthorized access attempts (unauthorizedReqLog.txt)
- Telegram alerts for critical events
- Geo-location caching (1 hour TTL)

9. **Network Security**

- Trust proxy: 1 (for Nginx/Render)
- Real IP detection (X-Forwarded-For, X-Real-IP)
- Public IP filtering
- Compression for response optimization

10. **Intrusion Detection**

- refreshOnRequest middleware (automatic token refresh)
- optionalJWT middleware (flexible authentication)

- secureMiddleware for request sanitization
- Fingerprint validation on token use

🚀 Deployment

Production Deployment (Render)

The backend is currently deployed on Render at `https://incubator.onrender.com`

Important Production Settings:

1. **Environment Variables** - Set all required env vars in Render dashboard
2. **NODE_ENV** - Must be set to `production`
3. **Trust Proxy** - Already configured: `app.set("trust proxy", 1)`
4. **Database** - Use production PostgreSQL database URL
5. **CORS** - Add production frontend URL to whitelist in `/backend/config/allowedOrigins.js`

Build Command (Backend):

```
npm install && npx prisma generate
```

Start Command (Backend):

```
npm start
```

Frontend Deployment

Build Command:

```
cd frontend && npm install && npm run build
```

Preview Build:

```
npm run preview
```

The build output will be in `frontend/dist/` directory.

Database Migration in Production

```
# Apply migrations
npx prisma migrate deploy
```

```
# Generate Prisma Client
npx prisma generate
```

⚠️ **Never use `prisma migrate reset` in production!**

Health Check Endpoints

Monitor application health:

```
GET /api/auth/status  # Check if API is responding
GET /api/faculties    # Check database connectivity
```

Performance Optimization

- **Compression**: Enabled via compression middleware
- **Response Size Limit**: JSON body limited to 150KB
- **Static Files**: Served via express.static from /public
- **Database Connection**: Pooled via Prisma
- **Rate Limiting**: Prevents DDoS attacks
- **Caching**: Geo-location cache (1 hour TTL)

Logging System

The application maintains four separate log files in `/backend/logs/`:

1. **reqLog.txt** - All HTTP requests with timestamps, methods, URLs, and origins
2. **errLog.txt** - Application errors with stack traces and context
3. **securityLog.txt** - Security-related events (rate limiting, suspicious activity)
4. **unauthorizedReqLog.txt** - Unauthorized access attempts with IP addresses

Log Format:

```
[YYYY-MM-DD HH:mm:ss] [METHOD] URL - Origin: origin - Message
```

Monitoring via Telegram:

Real-time alerts are sent to Telegram for:

- Rate limit breaches (DDoS attempts)
- Login attempt limit exceeded
- Registration spam attempts
- Server startup/shutdown (production only)
- Critical errors

Alert Format:

```

```

```
⚠️ Event Type
Details: ...
IP: xxx.xxx.xxx.xxx
Location: City, Country
User Agent: Browser/Version
```

🐛 Troubleshooting

Common Issues

Database Connection Error:

```
Error: Can't reach database server
```

Solution: Check PostgreSQL service and DATABASE_URL in .env

CORS Error:

```
Access-Control-Allow-Origin blocked
```

Solution: Verify FRONTEND_URL in backend .env matches frontend URL

JWT Token Expired:

```
401 Unauthorized
```

Solution: Token refresh should happen automatically, check cookie settings

Port Already in Use:

```
EADDRINUSE: Port 3000 already in use
```

Solution: Kill process or change PORT in .env

n8n Webhook Connection Error:

```
❌ AI Chat Error: connect ECONNREFUSED
```

Solution:

- Verify N8N_CHAT_WEBHOOK_URL is correct
- Check n8n instance is running
- Verify N8N_API_KEY matches your n8n configuration

Telegram Notifications Not Working:

Solution:

- Verify TELEGRAM_BOT_PRINTYDZ_NOTIFIER token is correct
- Check TELEGRAM_CHAT_ID is valid
- Ensure bot has permission to send messages to chat

Device Fingerprint Missing:

```
Warning: Fingerprint failed to load
```

Solution: This is a warning only; authentication will still work but with reduced security

Prisma Migration Errors:

```
Error: Can't drop table ... because it has foreign key constraints
```

Solution: Use `npx prisma migrate reset` to reset database (development only)

💡 Developer Notes

Important Code Patterns

1. Always use device fingerprint in authenticated requests:

```
// Fingerprint is automatically added by Axios interceptor
const response = await api.post("/auth/login", { email, password });
```

2. JWT is managed automatically:

```
// No need to manually handle access tokens
// refreshOnRequest middleware generates them automatically
// Just use api client with cookies enabled
```

3. Role checking in Vue components:

```
const userRole = computed(() => {
  if (!authStore.user?.role) return 0;
```

```
  return Array.isArray(authStore.user.role)
    ? Math.max(...authStore.user.role)
    : authStore.user.role;
});

const isSuperAdmin = computed(() => userRole.value >= 100);
const isAdmin = computed(() => userRole.value >= 90);
```

4. Error handling pattern:

```
try {
  const response = await api.post("/endpoint", data);
  // Success handling
} catch (err) {
  const errorMessage =
    err.response?.data?.message ||
    err.response?.data?.error ||
    "Operation failed";
  console.error(errorMessage);
}
```

5. Prisma transaction pattern:

```
const result = await prisma.$transaction(async (tx) => {
  const registration = await tx.project_registrations.create({ data });
  const students = await tx.registration_students.createMany({ data });
  return { registration, students };
});
```

Common Development Tasks

Reset database (development only):

```
npx prisma migrate reset
npx prisma generate
```

View database in Prisma Studio:

```
npx prisma studio
```

Check database schema:

```

```

```
npx prisma db pull  # Pull schema from database
npx prisma format   # Format schema.prisma
```

Generate new migration:

```
npx prisma migrate dev --name migration_name
```

View logs:

```
# Backend logs
tail -f backend/logs/reqLog.txt
tail -f backend/logs/errLog.txt
tail -f backend/logs/securityLog.txt

# Clear logs
> backend/logs/reqLog.txt
```

Test API endpoints:

```
# Using curl with cookies
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password"}' \
  -c cookies.txt

curl http://localhost:3000/api/users/me -b cookies.txt
```

Code Style Guidelines

- Use arrow functions for middleware and controllers
- Use async/await instead of promises
- Use const for all variables unless reassignment needed
- Use template literals for strings
- Use optional chaining: `user?.role?.[0]`
- Use nullish coalescing: `value ?? default`
- Keep functions small and focused
- Add comments for complex logic
- Use JSDoc for function documentation

Security Best Practices

- ✅ Never log passwords or tokens
- ✅ Always validate user input on server side

- ✅ Use parameterized queries (Prisma handles this)
- ✅ Sanitize HTML content (DOMPurify)
- ✅ Set rate limits on sensitive endpoints
- ✅ Use HTTPS in production
- ✅ Keep dependencies updated
- ✅ Use environment variables for secrets
- ❌ Never commit .env files
- ❌ Never disable CORS in production
- ❌ Never trust client-side validation alone

Testing the Application

Manual Testing Workflow:

1. **Public Access:**
   - Visit homepage at http://localhost:5173
   - Test AI chat without login
   - Submit project registration
   - Verify form validation errors
2. **Authentication:**
   - Login with test credentials
   - Test "Remember Me" functionality
   - Verify automatic token refresh
   - Test logout
3. **Role-Based Access:**
   - Login as SuperAdmin (role: 100)
   - Access all dashboard pages
   - Test CRUD operations on all entities
   - Login as Admin (role: 90)
   - Verify limited access (no faculty management)
4. **API Testing:**

```
# Get faculties (public)
curl http://localhost:3000/api/faculties

# Login
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@test.com","password":"password"}' \
  -c cookies.txt

# Get current user (authenticated)
curl http://localhost:3000/api/users/me -b cookies.txt
```

Database Seeding (for testing):

```
-- Create test SuperAdmin user
INSERT INTO users (email, phone_number, password, first_name, last_name,
role, status)
VALUES (
  'admin@ubma.dz',
  '0555000000',
  '$2b$10$hashgoeshere', -- Replace with actual bcrypt hash
  'Admin',
  'Test',
  ARRAY[100],
  'active'
);

-- Create test faculty
INSERT INTO faculties (faculty_name, faculty_name_en)
VALUES ('كلية العلوم الدقيقة', 'Faculty of Exact Sciences');

-- Create test department
INSERT INTO departements (departement_name, departement_name_en,
faculty_id)
VALUES ('قسم الإعلام الآلي', 'Computer Science', 1);
```

Generate bcrypt hash for testing:

```
const bcrypt = require("bcrypt");
const hash = await bcrypt.hash("your-password", 10);
console.log(hash);
```

🤔 Technology Decisions

Why These Technologies?

Backend:

- **Express.js 5.x**: Latest version with improved performance and TypeScript support
- **Prisma ORM**: Type-safe database queries, excellent PostgreSQL support, automatic migrations
- **PostgreSQL**: Robust relational database, perfect for complex relationships (jury, projects, users)
- **bcrypt 6.x**: Industry standard for password hashing with adjustable salt rounds
- **JWT**: Stateless authentication, scalable, works well with REST APIs
- **n8n Integration**: Flexible AI workflow automation, self-hosted option, easy to customize

Frontend:

- **Vue 3**: Composition API provides better TypeScript support and code organization
- **Pinia**: Modern Vue state management, lightweight, intuitive API

- **Vite**: Lightning-fast dev server, optimized production builds
- **FingerprintJS**: Reliable device identification for enhanced security

Security:

- **Helmet.js**: Comprehensive HTTP header security out of the box
- **Express Rate Limit**: Flexible rate limiting with multiple strategies
- **DOMPurify**: Best-in-class XSS protection for user-generated content

Monitoring:

- **Telegram Notifications**: Real-time alerts without complex infrastructure
- **File-based Logging**: Simple, reliable, easy to analyze and rotate

📝 License

This project is proprietary and confidential. Unauthorized copying, distribution, or use is strictly prohibited.

© 2025 Université Badji Mokhtar Annaba (UBMA), Algeria. All rights reserved.

📞 Support

For technical support or questions:

- **Email**: youcef-almahdi.berbague@univ-annaba.dz
- **Phone**: +213771282136
- **Address**: Sidi Ammar, Annaba, Algeria

👨‍💻 Author

Berbague Youcef Al Mahdi

- GitHub: @Youcef-Berbague
- Repository: incubator

Last Updated: December 14, 2025
Version: 1.0.0
Maintainer: Berbague Youcef Al Mahdi